

Notes on Distributional Reinforcement Learning

Preliminaries

These notes follow the modern treatment of distributional reinforcement learning in [BDR23].

1 Linear Function Approximation

Let $\mathcal{X}$ be the state space and $\mathcal{A}$ the action space. At time t , the agent is in state $X_t = x$, selects an action $A_t = a$, receives a reward R_t , and moves to a next state X_{t+1} . A policy π maps states to distributions over actions, and a discount factor $\gamma \in [0, 1)$ weights future rewards.

We first discuss linear approximation for the state-value function

$$V^\pi : \mathcal{X} \rightarrow \mathbb{R}.$$

This is the cleanest setting because the input is only the state. Starting from $X_0 = x$ and then following π , define the discounted return

$$G^\pi(x) = \sum_{t=0}^{\infty} \gamma^t R_t.$$

The state-value function is the mean return:

$$V^\pi(x) = \mathbb{E}_\pi [G^\pi(x)].$$

It is the fixed point of the Bellman operator $\mathcal{T}^\pi$, defined by

$$(\mathcal{T}^\pi V)(x) = \mathbb{E}_{A \sim \pi(\cdot|x)} [\mathbb{E} [R + \gamma V(X') \mid X = x, A]].$$

The Bellman equation is $V^\pi = \mathcal{T}^\pi V^\pi$.

In a small finite state space, V^π can be stored as a table with one number per state. Function approximation replaces this table by a parametric family. In linear function approximation we fix a feature map

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^d, \quad \phi(x) = \begin{pmatrix} \phi_1(x) \\ \vdots \\ \phi_d(x) \end{pmatrix}.$$

The approximate value function is

$$V_w(x) = \phi(x)^\top w, \quad w \in \mathbb{R}^d.$$

The vector w is shared across all states. Learning from one transition can therefore change predictions at other states with similar features.

For a sampled transition (x_t, r_t, x'_t) , the TD error is

$$\delta_t = r_t + \gamma V_{w_t}(x'_t) - V_{w_t}(x_t).$$

Since $V_w(x) = \phi(x)^\top w$, the semi-gradient TD update is

$$w_{t+1} = w_t + \alpha_t \delta_t \phi(x_t).$$

The scalar target behind this update is

$$r_t + \gamma V_{w_t}(x'_t),$$

so TD learning can be read as repeatedly making the current prediction $V_{w_t}(x_t)$ more consistent with a one-step Bellman target.

The linear family is

$$\mathcal{V}_\phi = \{V_w : V_w(x) = \phi(x)^\top w, w \in \mathbb{R}^d\}.$$

Usually $V^\pi \notin \mathcal{V}_\phi$. The Bellman update $\mathcal{T}^\pi V_w$ may leave the linear family, so the approximate problem is often understood through a projected Bellman equation

$$V_w = \Pi \mathcal{T}^\pi V_w,$$

where Π projects back onto $\mathcal{V}_\phi$ in a norm determined by the sampling distribution. The representation ϕ is therefore not a cosmetic choice: it determines which Bellman fixed points are even expressible.

The same idea works for action-values

$$Q^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}.$$

Now the feature map takes a state-action pair,

$$\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^d,$$

and the approximation is

$$Q_w(x, a) = \phi(x, a)^\top w.$$

The TD error for policy evaluation becomes

$$\delta_t = r_t + \gamma Q_{w_t}(x'_t, a'_t) - Q_{w_t}(x_t, a_t), \quad a'_t \sim \pi(\cdot \mid x'_t),$$

with update

$$w_{t+1} = w_t + \alpha_t \delta_t \phi(x_t, a_t).$$

We will use this action-value version below because QR-DQN and the deep distributional algorithms are control methods: the agent must compare actions in the same state, and this comparison is immediate from $Q(x, a)$.

2 Distributional Value Approximation

Distributional reinforcement learning replaces the scalar value function by a return-distribution function. Instead of storing only the mean $Q^\pi(x, a) = \mathbb{E}[G^\pi(x, a)]$, we keep the law of the return:

$$\eta^\pi(x, a) = \text{Law}(G^\pi(x, a)).$$

Thus

$$\eta^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{P}(\mathbb{R}),$$

where $\mathcal{P}(\mathbb{R})$ denotes probability distributions on the real line. The classical action-value function is recovered by taking expectations:

$$Q^\pi(x, a) = \int_{\mathbb{R}} z \eta^\pi(x, a)(dz).$$

The distributional Bellman operator is most cleanly written using pushforwards. For $r \in \mathbb{R}$, define the affine

map

$$f_{r,\gamma}(z) = r + \gamma z.$$

If $\nu \in \mathcal{P}(\mathbb{R})$, then $(f_{r,\gamma})_{\#}\nu$ is the distribution of $r + \gamma Z$ when $Z \sim \nu$. The distributional Bellman operator $\mathcal{T}^\pi$ maps a return-distribution function η to

$$(\mathcal{T}^\pi \eta)(x, a) = \mathbb{E}[(f_{R,\gamma})_{\#}\eta(X', A') \mid X = x, A = a], \quad A' \sim \pi(\cdot \mid X').$$

The expectation averages over the random reward, the next state, and the next action sampled from the policy. The distributional Bellman equation is

$$\eta^\pi = \mathcal{T}^\pi \eta^\pi.$$

Taking means on both sides recovers the usual Bellman equation for Q^π , but the distributional equation is stronger: it describes the whole random return, not only its expectation.

Just as linear TD restricts Q to a class $\mathcal{Q}_\phi$, distributional methods restrict η to a class of approximate distributions. The quantile family is one of the most important such choices. For a distribution ν with cumulative distribution function F_ν , define its quantile function by

$$F_\nu^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_\nu(z) \geq \tau\}, \quad \tau \in (0, 1).$$

Choose N quantile levels, usually the midpoints

$$\tau_i = \frac{2i-1}{2N}, \quad i = 1, \dots, N.$$

An N -quantile approximation represents $\eta(x, a)$ by the empirical distribution

$$\eta_\theta(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a)},$$

where δ_z is a Dirac mass at z . The parameter $\theta_i(x, a)$ is interpreted as the predicted τ_i -quantile of the return distribution at (x, a) .

The direct distributional analogue of linear action-value approximation is to make each quantile location linear in the same feature vector:

$$\theta_i(x, a) = \phi(x, a)^\top w_i, \quad w_i \in \mathbb{R}^d.$$

Equivalently, with $W = (w_1, \dots, w_N) \in \mathbb{R}^{d \times N}$,

$$\eta_W(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\phi(x, a)^\top w_i}.$$

The scalar approximation $Q_w(x, a)$ has become a measure-valued approximation $\eta_W(x, a)$. The same features are now asked to explain not only the average return, but also the spread, asymmetry, and tail behavior of the return.

3 Quantile TD

Quantile TD applies temporal-difference learning to the quantile parameterization. For policy evaluation, sample a transition and next action

$$(x_t, a_t, r_t, x'_t, a'_t), \quad a'_t \sim \pi(\cdot \mid x'_t).$$

The distributional Bellman target is represented by the N target atoms

$$y_j = r_t + \gamma \theta_j(x'_t, a'_t), \quad j = 1, \dots, N.$$

The current prediction at (x_t, a_t) has atoms

$$z_i = \theta_i(x_t, a_t), \quad i = 1, \dots, N.$$

Quantile regression uses the asymmetric absolute loss

$$\rho_\tau(u) = u(\tau - \mathbb{1}_{\{u < 0\}}).$$

The empirical Quantile TD loss is then

$$L(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}(y_j - z_i).$$

The residual $y_j - z_i$ compares an atom of the Bellman target distribution with the predicted τ_i -quantile at the current state-action pair. Minimizing this loss moves $\eta_W(x_t, a_t)$ toward

$$\frac{1}{N} \sum_{j=1}^N \delta_{r_t + \gamma \theta_j(x'_t, a'_t)} = (f_{r_t, \gamma})_{\#} \eta_W(x'_t, a'_t).$$

For linear quantiles, a subgradient update for each head is

$$w_{i,t+1} = w_{i,t} + \alpha_t \left(\frac{1}{N} \sum_{j=1}^N (\tau_i - \mathbb{1}_{\{y_j < z_i\}}) \right) \phi(x_t, a_t), \quad i = 1, \dots, N.$$

This is the exact analogue of the semi-gradient TD update:

$$\delta_t = r_t + \gamma Q_{w_t}(x'_t, a'_t) - Q_{w_t}(x_t, a_t) \rightsquigarrow \{y_j - z_i\}_{i,j=1}^N.$$

The scalar TD target is replaced by a Bellman target distribution, and the single scalar TD error is replaced by all pairwise quantile residuals.

In implementations, one often replaces ρ_τ by a quantile Huber loss. This is the smooth loss used in QR-DQN [Dab+18a]. Define

$$\mathcal{L}_\kappa(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa(|u| - \frac{1}{2}\kappa), & |u| > \kappa, \end{cases}$$

and

$$\rho_\tau^\kappa(u) = |\tau - \mathbb{1}_{\{u < 0\}}| \frac{\mathcal{L}_\kappa(u)}{\kappa}.$$

The corresponding loss is

$$L_\kappa(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^\kappa(y_j - z_i).$$

The Huber version keeps the asymmetric quantile penalty but smooths the non-differentiability around zero, which is useful when the quantile locations are produced by a neural network.

For control, the next action is usually selected greedily with respect to the mean of the predicted distribution:

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a').$$

The target atoms become

$$y_j = r + \gamma \theta_j(x', a^*(x')), \quad j = 1, \dots, N.$$

This is the quantile analogue of replacing policy evaluation TD by Q-learning.

4 Gateway to Deep Representations

The linear approximation above separates representation and prediction:

fixed features $\phi(x, a)$ and linear heads w_i .

This is the useful gateway into deep distributional reinforcement learning. The deep version keeps the same conceptual decomposition, but replaces the fixed feature map by a learned representation. A neural network maps the observation or state to a feature vector

$$\psi_\xi(x) \in \mathbb{R}^d,$$

and the final layers produce a distributional prediction. In the quantile case one may write

$$\theta_i(x, a; \omega) = f_i(\psi_\xi(x), a), \quad i = 1, \dots, N.$$

The return-distribution approximation becomes

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a; \omega)}.$$

The parameter ω includes both the representation parameters ξ and the prediction heads. Linear Quantile TD is recovered as the special case in which ψ_ξ is fixed and f_i is linear. Deep quantile methods instead optimize all of ω by backpropagating through the distributional TD loss.

5 QR-DQN

QR-DQN is the deep control version of Quantile TD [Dab+18a]. It uses a neural network to output N quantile locations for each action:

$$\theta_i(x, a; \omega), \quad i = 1, \dots, N, \quad a \in \mathcal{A}.$$

For each state-action pair, the predicted return distribution is

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a; \omega)}.$$

The predicted scalar action-value is the mean of this empirical distribution:

$$Q_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \theta_i(x, a; \omega).$$

Thus QR-DQN can still choose greedy actions in the usual value-based way, but the value used for action selection is obtained by averaging the learned quantiles.

As in DQN, QR-DQN uses two sets of parameters. The online network has parameters ω and is updated by gradient descent. The target network has parameters $\bar{\omega}$ and is held fixed for several updates before being copied from ω . For a transition

$$(x, a, r, x', d),$$

where $d \in \{0, 1\}$ indicates termination, the greedy next action is selected by the mean of the target-network

distribution:

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a'; \bar{\omega}).$$

The target quantile atoms are

$$y_j = r + \gamma(1 - d)\theta_j(x', a^*(x'); \bar{\omega}), \quad j = 1, \dots, N.$$

The online prediction atoms for the sampled action are

$$z_i = \theta_i(x, a; \omega), \quad i = 1, \dots, N.$$

QR-DQN minimizes the pairwise quantile Huber loss

$$L_{\text{QR}}(\omega) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^{\kappa}(y_j - z_i),$$

with quantile levels

$$\tau_i = \frac{2i - 1}{2N}.$$

In a minibatch $\mathcal{B}$ sampled from replay memory, the objective is

$$\mathcal{L}_{\text{QR}}(\omega) = \frac{1}{|\mathcal{B}|} \sum_{(x, a, r, x', d) \in \mathcal{B}} L_{\text{QR}}(\omega; x, a, r, x', d).$$

The target atoms are treated as constants when differentiating with respect to ω ; this is the usual semi-gradient idea inherited from TD learning and DQN.

The quantile Huber loss is important because the raw quantile loss $\rho_{\tau}(u)$ is non-smooth at zero. QR-DQN uses

$$\mathcal{L}_{\kappa}(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa(|u| - \frac{1}{2}\kappa), & |u| > \kappa, \end{cases}$$

and

$$\rho_{\tau}^{\kappa}(u) = |\tau - \mathbb{1}_{\{u < 0\}}| \frac{\mathcal{L}_{\kappa}(u)}{\kappa}.$$

For small residuals, the loss has a squared-error shape, which gives smoother gradients. For large residuals, it grows linearly, which prevents very large TD errors from dominating the update. The factor $|\tau - \mathbb{1}_{\{u < 0\}}|$ keeps the asymmetry needed for quantile regression: underestimation and overestimation are penalized differently depending on the quantile level τ .

How QR-DQN Relates to DQN

DQN learns one scalar per action:

$$x \mapsto \{Q_{\omega}(x, a) : a \in \mathcal{A}\}.$$

QR-DQN learns a collection of quantiles per action:

$$x \mapsto \{\theta_i(x, a; \omega) : i = 1, \dots, N, a \in \mathcal{A}\}.$$

The scalar Bellman target

$$r + \gamma \max_{a'} Q_{\bar{\omega}}(x', a')$$

is replaced by the empirical target distribution

$$\frac{1}{N} \sum_{j=1}^N \delta_{r+\gamma(1-d)\theta_j(x', a^*(x'); \bar{\omega})}.$$

The squared TD loss is replaced by the pairwise quantile Huber loss. Action selection remains risk-neutral unless we deliberately change it: QR-DQN chooses actions by the mean of the predicted quantiles. The learned object, however, is much richer than the mean.

Representation Learning Signal

The representation $\psi_\xi(x)$ in QR-DQN is trained by many losses at once. For a single transition, the network sees all residuals

$$y_j - \theta_i(x, a; \omega), \quad i, j = 1, \dots, N.$$

Lower quantile heads receive large gradients when the lower tail is poorly placed; upper quantile heads receive large gradients when the upper tail is poorly placed. Thus the shared representation is shaped not only by expected return, but by the whole geometry of the target return distribution.

This gives a useful interpretation of the empirical strength of distributional methods. A scalar DQN representation only needs to make the conditional mean easy to predict. A QR-DQN representation must make many parts of the return distribution easy to predict simultaneously. The extra prediction problem can act as a richer training signal, even when the final policy is still selected by the mean.

6 Beyond Fixed Quantile Locations

QR-DQN fixes the quantile levels $\tau_1, \dots, \tau_N$. Implicit Quantile Networks (IQN) generalize this idea by learning a function of the quantile level itself [Dab+18b]. Instead of outputting only a finite list of quantile locations, IQN models

$$(x, a, \tau) \mapsto Z_\omega(x, a; \tau), \quad \tau \in (0, 1).$$

During training, quantile levels are sampled, for example

$$\tau_i, \tau'_j \sim \mathcal{U}(0, 1),$$

and the same quantile Huber principle is applied to residuals

$$r + \gamma(1 - d)Z_{\bar{\omega}}(x', a^*; \tau'_j) - Z_\omega(x, a; \tau_i).$$

This makes the quantile function itself part of the model. The network must learn a representation of both the state and the quantile level. Conceptually, IQN moves from a finite set of quantile heads to a continuous return-distribution model.

This is the natural gateway to deep representations. Chapter-level questions about deep distributional RL can be phrased as:

What representation makes $\eta^\pi(x, a)$ easy to approximate?

Linear Quantile TD answers this with fixed ϕ and linear quantile heads. QR-DQN replaces ϕ by a learned deep representation and keeps finitely many quantile heads. IQN goes one step further and makes the quantile level an input, so that the representation must support an entire quantile function. In all cases, the core idea is the same: approximate the distributional Bellman fixed point, not merely its expectation.

References

- [BDR23] Marc G. Bellemare, Will Dabney, and Mark Rowland. *Distributional Reinforcement Learning*. MIT Press, 2023.
- [Dab+18a] Will Dabney et al. “Distributional Reinforcement Learning with Quantile Regression”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2018.
- [Dab+18b] Will Dabney et al. “Implicit Quantile Networks for Distributional Reinforcement Learning”. In: *Proceedings of the 35th International Conference on Machine Learning*. 2018, pp. 1096–1105.